

2026 EDITION • FREE

THE BEGINNER'S GUIDE TO AI SECURITY

A Field Manual — From Cyber GRC to AI Security

How you and AI can be secure together. Shaken, not stirred.



KAMO TUNE AI

A working field manual — shareable with attribution

Welcome to the Bureau.

This manual is for security, compliance, and governance professionals extending their expertise into AI security — and for anyone entering the field who wants to see how the pieces connect.

It follows one throughline: **the skills that make someone effective in traditional security and GRC are the same skills that secure AI systems — but only if you can implement the controls, not just design them.**

How to use this manual

Each section builds on the last — study in order. Every section ends with a self-test worksheet. The rule: if you can't answer out loud without notes, re-read. Recognition is not recall.

Meet your handler



Agent A, at your service. I'm a non-human identity with a martini and — until you finish Section 5 — probably too many permissions. Let's fix that together.

Suggested pacing

Sections 1–4 in one sitting or day. Sections 5–7 in a second. Section 8 is an ongoing lab track — reading builds vocabulary, labs build fluency.

Contents

00	Know Your Subject: AI — models, agents & autonomy	5
01	Security Fundamentals — the CIA Triad & friends	10
02	The GRC Layer — governance, risk & compliance	15
03	Cloud Security Foundations	20
04	Infrastructure as Code & Policy as Code	25
05	Identity — human, machine & agent	30
06	AI Governance — the frameworks layer	37
07	AI Security — the threat layer	42
08	The Lab Track	48
§	Scope, Limitations & How to Read Claims	50
†	Sources & Further Reading	51
★	The One-Paragraph Summary	52

The thesis of this manual

Four of the seven layers in the agentic defense stack are classic infrastructure controls. The road to AI security runs *through* the fundamentals, not around them.

The Line You're Walking

Every section is one step on a single line — from control objectives on paper to attested agent identities in production.

01 · Fundamentals — the CIA triad, least privilege, defense in depth. The physics of the discipline.

02 · GRC — objectives, tests, evidence, findings. The *grammar* of security.

03 · Cloud — where identity replaces the network as the perimeter.

04 · IaC & Policy as Code — where controls become executable and preventive.

05 · Identity — humans, machines, and the coming wave of agents.

06 · AI Governance — what must be true, and how it's proven.

07 · AI Security — what attackers will try, from poisoned data to hijacked agents.

08 · The Lab Track — where vocabulary becomes fluency.

Cyber GRC → AI Governance → AI Security

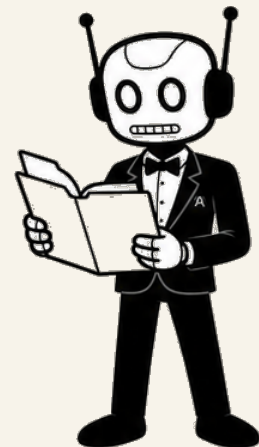
Know Your Subject: AI

Before securing a thing, define the thing. What AI is, what agents are, and what they can actually do.

00

- What AI actually is (and isn't)
- Anatomy of an agent — six components
- Action-space, autonomy & multi-agent patterns
- Self-test worksheet

You can't threat-model what you can't define. Start here.



What AI Actually Is

Machine learning, in one breath

Instead of programming explicit rules, you show a system many examples and it learns statistical patterns. The output is a **model** — a mathematical function tuned by training data — not a program a human wrote line by line.

Large language models (LLMs)

Models trained on vast text corpora to predict the next token. That simple objective, at scale, yields systems that can reason, plan, write, and code. Small, large, and multimodal variants (SLM / LLM / MLLM) differ in size and input types — the principle is the same.

CORE CONCEPT The property that changes everything: probabilistic behavior

A firewall rule either blocks or doesn't. A model's behavior is a **distribution** — the same input can yield different outputs, and no input has a guaranteed output. Testing, acceptance criteria, and monitoring must all become statistical. This single property drives most of what's genuinely new in Sections 6 and 7.

Agent A says

An LLM on its own mostly just produces output — text, images, structured data. It becomes interesting — and dangerous — when you give it the ability to **act**. Turn the page.

Anatomy of an Agent

Agentic AI systems plan across multiple steps to achieve specified objectives. There's no single agreed definition, but production agents share six components:⁸

COMPONENT	WHAT IT DOES
1 · Model	The reasoning and planning engine — the "brain." Interprets input, decides what to do.
2 · Instructions	Natural-language commands defining role, capabilities, and behavioral constraints (the system prompt).
3 · Memory	Short- or long-term stored information — prior interactions, retrieved knowledge. (Section 7: a persistent attack surface.)
4 · Planning & reasoning	The ability to output and follow a series of steps toward a goal.
5 · Tools	What lets the agent <i>act</i> : writing to files and databases, calling APIs, performing transactions.
6 · Protocols	Standardized connections — MCP for agent-to-tool, A2A for agent-to-agent.

The security preview

Read that table again as an attacker: instructions can be overridden, memory can be poisoned, tools can be misused, protocols can be spoofed. Every component is a surface. That's Section 7's whole outline.

Two Dials: Action-Space & Autonomy

What an agent *can do* is set by two independent dials — and governance is mostly the art of setting them deliberately.

Dial 1 — Action-space (authority)

The range of actions the agent is permitted to take: sandboxed tools only → internal systems → external systems; read-only → write. A **computer-use agent** — whose tool is a computer and browser — can take nearly any action a human can, which dramatically widens this dial.

Dial 2 — Autonomy

How much the agent decides on its own. **Supervised:** human approves each action. **Semi-autonomous:** agent executes routine actions, human reviews flagged items. **Fully autonomous:** agent plans, executes, and iterates alone — requiring deterministic enforcement, budget limits, and a kill switch.⁷

The governance question

Not just "what type of agent is this?" but "**what can this agent do without a human confirming the action?**" Risk scales with the blast radius of unsupervised action — the window for human detection narrows as autonomy rises.

When agents multiply

Multi-agent systems arrange specialists in patterns: **sequential** (each agent's output feeds the next), **supervisor** (one coordinator directs specialists), and **swarm** (parallel agents handing off as needed). Each pattern multiplies identities, delegation chains, and trust boundaries — the exact terrain of Sections 5 and 7.



Worksheet 00 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1** Name the six components of an agent — and one attack surface each component creates.

- 2** Explain action-space vs. autonomy. Why are they independent dials?

- 3** Why does probabilistic model behavior matter more for security than for ordinary software quality?

- S** **Scenario:** A vendor demos an agent that reads your inbox and books travel on a corporate card. Using the two dials, describe its action-space and autonomy — and name the one question you'd ask before approving deployment.

Bureau rule: you can't threat-model what you can't define. Definitions first, exploits later.

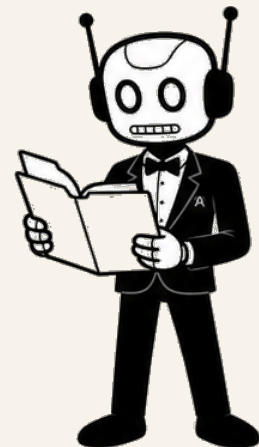
Security Fundamentals

The concepts everything else stands on. New technology, same triad.

01

- The CIA Triad — and how AI attacks each leg
- Authentication vs. authorization
- Least privilege & defense in depth
- Risk, threat, vulnerability, exploit
- Self-test worksheet

Before we secure the robots, we secure the basics.



The CIA Triad

Three properties every security control ultimately protects. Every threat in this manual attacks at least one of them.

Confidentiality

Information is accessible only to those authorized to see it. *Controls:* encryption, access control, data classification. *AI-era attack:* model extraction — querying a model until you've stolen a working copy.

Integrity

Information and systems are accurate, and altered only by authorized change. *Controls:* hashing, signatures, change management, input validation. *AI-era attack:* prompt injection — the system behaves in unauthorized ways.

Availability

Systems and data are accessible when needed. *Controls:* redundancy, backups, capacity planning. *AI-era attack:* resource exhaustion against inference endpoints.

Agent A says

Memorize the pattern, not just the triad: **new asset class, same three properties**. When you meet an unfamiliar AI threat, your first question is "which leg does this attack?"

AuthN, AuthZ & Least Privilege

Authentication (AuthN) — who are you?

Proving an identity: passwords, passkeys, certificates, biometrics. The bouncer checking ID at the door.

Authorization (AuthZ) — what may you do?

Determining permissions once identity is established. The wristband that says which rooms you may enter.

They are **separate questions with separate failure modes** — and confusing them is the root of an enormous number of real breaches. A perfectly authenticated identity can still be catastrophically over-authorized.

Least privilege

Every identity — human or machine — holds only the permissions required for its task, and no more. **The single most transferable control in this manual.** It appears in SOC 2 audits, IAM policy reviews, and agent tool-scoping alike — identical objective, different mechanism.

Defense in depth

No single control is trusted to hold. Layer controls so that when one fails — and one will — the others contain the damage. "We have a firewall" is a control; it is not a strategy.

Thinking in Risk

Risk = likelihood × impact

Security is not about eliminating risk (impossible), but identifying, prioritizing, and treating it: **accept, mitigate, transfer, or avoid.**

TERM	MEANING
Threat	A potential source of harm — an attacker, an insider, a failure mode.
Vulnerability	A weakness the harm could travel through — a misconfiguration, a flaw, a gap.
Exploit	The act (or tool) that turns one into the other.

Non-repudiation & auditability

Actions must be attributable to identities, with evidence that can't be quietly altered. Logging isn't bureaucracy — it's how accountability becomes *possible*. Hold this thought; it returns as the backbone of AI governance in Section 6.

Field note

When an AI agent takes a consequential action, "who did this, under whose authority, and can we prove it?" is a non-repudiation question. The oldest concept in this section is also the newest problem in the field.



Worksheet 01 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1 Name one control for each leg of the CIA triad — and one AI-era threat that attacks each leg.

- 2 Explain the difference between authentication and authorization using a non-technical analogy.

- 3 Why is "we have a firewall" not a defense-in-depth strategy?

- S** **Scenario:** A website chatbot can query the order database and email customers. Which CIA legs are exposed, and what is the least-privilege question to ask about its access?

Bureau rule: if you can't answer without notes, re-read the section. Recognition is not recall.

The GRC Layer

Governance, Risk & Compliance — where security intentions become verifiable practice.

02

- Every security problem is a controls problem
- ISO 27001, SOC 2 & NIST CSF
- The concepts that transfer straight to AI
- Self-test worksheet

Paperwork? No. This is the grammar of the whole discipline.



The Four Questions

The productive question is never "is this system safe?" It's four questions:

1. What control objective exists?

The outcome that must be true — e.g., "only authorized identities access production data."

2. What's the test procedure?

How you'd check it — inspect the policy, sample the access list, attempt the forbidden action.

3. What's the evidence?

The artifact that proves it — logs, configs, reports, screenshots. A control without evidence might as well not exist.

4. What's the pass/fail outcome?

A finding, stated plainly, with severity and remediation.

Why this transfers

Professionals who internalize this framing can evaluate **any** system — including AI systems they've never seen before. AI changes the vocabulary; the grammar stays.

Three Frameworks to Know Cold

ISO 27001

The certifiable information security management system (ISMS) standard. Plan-Do-Check-Act, risk assessment driving control selection, the Annex A catalog, internal audit, management review. Its structure is a **template later standards reuse** — including the AI ones.

SOC 2

An attestation report (not a certification) against the Trust Services Criteria. **Type I** evaluates control *design* at a point in time; **Type II** evaluates *operating effectiveness* over a period. Designed-vs-actually-working is a mental model you will reuse constantly.

NIST Cybersecurity Framework

Six functions — Govern, Identify, Protect, Detect, Respond, Recover. The common vocabulary of enterprise security programs.

Control types — stop it, catch it, fix it

Preventive controls stop the bad thing; **detective** controls catch it; **corrective** controls fix it. Hold this taxonomy — Section 4 shows how modern infrastructure makes preventive controls automatable.

Objectives Outlive Technology

The key distinction

Control objectives vs. control activities. The objective is the outcome; the activity is the mechanism. Objectives are stable across decades; activities change with technology. AI changes the activities — not the objectives.

OBJECTIVE (STABLE)	2010 ACTIVITY	2026 ACTIVITY
Least privilege	AD group review	Agent tool-scope policy
Change control	CAB ticket	Model-swap approval & provenance check
Accountability	Server access logs	Agent action & delegation logs
Third-party risk	Vendor questionnaire	Model & dataset provenance review

Supply chain risk, previewed

Your risk posture includes every vendor, dependency, and component you consume. This becomes acute in AI, where models, datasets, and tools are all supplied by third parties. Section 7 returns here with teeth.



Worksheet 02 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1 What's the difference between a SOC 2 Type I and Type II report — and why does that distinction matter beyond auditing?

- 2 Give a control objective that would remain identical between a 2010 data center and a 2026 AI agent deployment.

- 3 Why is "the control exists in policy" insufficient as an audit answer?

- S** **Scenario:** Your team says the AI approval workflow "exists — it's documented in the wiki." Design the Type II test: what evidence would prove it actually operated over the last quarter?

Bureau rule: a control without evidence might as well not exist. Same goes for your knowledge.

Cloud Security Foundations

Nearly every AI system runs in the cloud. You cannot secure what runs on infrastructure you don't understand.

- The shared responsibility model
- IAM is the perimeter
- Cross-cloud translation & Zero Trust
- Self-test worksheet

My blast radius is exactly my IAM permissions. Yours too.



Whose Job Is It?

The provider secures the cloud. You secure what's in it.

Physical facilities, hypervisors, and core services belong to the provider. Identities, configurations, data, and workloads belong to **you**. Analyst research consistently attributes the overwhelming majority of cloud breaches to customer-side misconfiguration — not provider failure.¹

IAM is the perimeter

In cloud environments, network boundaries are soft; **identity is the real control plane**. The must-know primitives (AWS terms; translation table next page):

- **Roles vs. users:** roles are assumable identities issuing temporary credentials. Workloads use roles — never long-lived access keys in code or config.
- **Policies:** JSON documents — Effect / Action / Resource / Condition. Evaluation logic: *explicit deny* > *explicit allow* > *implicit deny*.
- **STS:** issues temporary credentials when a role is assumed — the mechanic behind workload identity federation.
- **Instance profiles & service-linked roles:** how compute gets an identity without embedded secrets.
- **Trust policy vs. permission policy:** one defines *who can assume* a role; the other defines *what it can do*. Two documents. Auditors confuse them constantly — don't.

Learn Once, Translate Everywhere

CONCEPT	AWS	AZURE	GCP
Assumable machine identity	IAM role	Service principal / managed identity	Service account
Compute identity, no secrets	Instance profile	Managed identity	SA + Workload Identity Fed.
Temporary credentials	STS	Entra token service	STS / WIF
Secrets storage	Secrets Manager	Key Vault	Secret Manager

CORE CONCEPT Zero Trust in one sentence

Never trust based on network location; authenticate and authorize **every request** based on identity, device, and context — and assume the network is hostile.

Why this matters for AI

An AI agent calling tools is a workload making API calls. Its blast radius is exactly its IAM permissions. "Agent security" without IAM rigor is theater.

The Misconfigurations You'll Actually Meet

The eternal access key

A long-lived key minted in 2021, embedded in a config file, never rotated, never owned. Replace with roles and temporary credentials — always.

The wildcard policy

*Action: * on Resource: ** because the demo was due Friday. Least privilege dies in deadline pressure; policy gates (Section 4) resurrect it.

The public bucket

Storage exposed to the internet by one checkbox. A customer-side failure in the shared responsibility model — every time.

The confused trust policy

A role assumable by far more principals than intended, because trust and permission policies were reviewed as one document instead of two.

Agent A says

Every one of these classic failures becomes an **agent** failure the moment an AI system inherits the identity. Same hole, faster traffic through it.



Worksheet 03 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1** Explain the difference between a trust policy and a permission policy.

- 2** Why are long-lived access keys an anti-pattern, and what replaces them?

- 3** In the shared responsibility model, whose failure is a publicly exposed storage bucket — and why?

- S** **Scenario:** To make a demo work, an agent's IAM role was copied from a human admin's. Walk the blast radius: name three actions the agent can now take that its task never required.

Bureau rule: identity is the perimeter. Check yours before checking anyone else's.

Infrastructure as Code

Where governance stops being a document and becomes a pipeline gate.

- Why IaC matters to governance people
- Terraform must-knows (state is the story)
- Policy as code — GRC, executable
- Self-test worksheet

Controls that run before deployment beat findings written after the breach.



Infrastructure Became Text

IaC turns infrastructure into reviewable, versionable, testable *text*. That means controls can be enforced **before deployment** — governance shifts left, from audit-after-the-fact to automated gates in the pipeline.

Terraform core concepts

- **Providers:** plugins that talk to platforms (AWS, Azure, Kubernetes). Declared and version-pinned.
- **Resources & data sources:** resources create and manage infrastructure; data sources read what already exists.
- **The workflow:** *init* → *plan* (compute the diff between config and reality) → *apply* (execute it) → *destroy*.
- **Modules:** reusable, parameterized bundles. A hardened module library is a preventive control — teams consume pre-approved patterns.
- **Drift:** when reality diverges from state (someone clicked in the console). *plan* detects it. Drift detection = continuous control monitoring, automated.

State — THE security topic

State files can contain **secrets in plaintext**. Remote backends with encryption, access control, and locking are mandatory in production. And *sensitive = true* masks output — it does **not** encrypt state.

GRC, Executable

This is where your controls matrix stops being prose and starts being enforced.

Terraform plan → JSON export → OPA / Rego rules → allow or deny

A Terraform plan exports as JSON. A policy engine — **OPA (Open Policy Agent)**, rules written in **Rego** — evaluates it: *"deny any storage bucket without encryption," "deny IAM policies granting Action: *."*

Why this is the whole GRC grammar in one mechanism

A failed policy check **blocks the deployment** (preventive), **produces an audit record** (evidence), and states **pass/fail** (finding). Objective, test, evidence, outcome — automated.

Names to recognize

Sentinel (HashiCorp's commercial engine); **Checkov** and **tfsec** (static scanners). Know they exist and what category they occupy.

Agent A says

The professional who can write the control objective *and* the Rego rule that enforces it occupies rare ground. That intersection is the point of this manual.

One Mechanism, Three Control Types

CONTROL TYPE	IAC MECHANISM	WHAT IT PROVES
Preventive	OPA gate on the plan	Non-compliant infra never deploys
Detective	Drift detection via <i>plan</i>	Reality still matches approved config
Corrective	Re-apply from source	Deviations are reverted, with history

Why AI systems make this urgent

AI deployments move fast and multiply identities, endpoints, and data paths. Manual review cannot keep pace with agentic change velocity — **policy-as-code is how governance keeps up with machine speed.** When Section 7 describes agents causing damage through legitimate permissions, the pipeline gate is one of the few controls fast enough to matter.

Field note

Continuous compliance platforms are built on exactly this loop: monitor configuration, compare to control objectives, evidence the result. IaC is the substrate that makes it possible.



Worksheet 04 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1** What can leak through Terraform state, and what are the mitigations?

- 2** Walk through how a Terraform JSON plan becomes an enforced policy gate.

- 3** Map preventive / detective / corrective onto: an OPA pipeline gate, drift detection, and a remediation apply.

- S** **Scenario:** A data scientist runs Terraform from a laptop with local state to ship a model endpoint before Friday. Name two risks in that sentence and the pipeline control that prevents each.

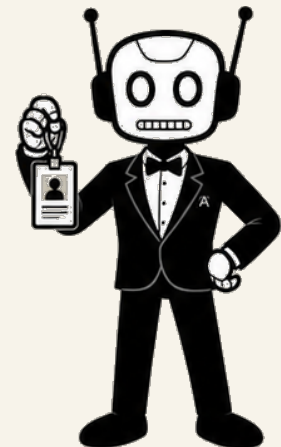
Bureau rule: if the control only exists in a document, it's a wish. Put it in the pipeline.

Identity

Human, machine — and the coming wave of agents. The connective tissue of this entire manual.

- The human identity stack (mature)
- The non-human identity stack (a mess)
- Secret zero & attested identity
- Five shifts & the credentials → identity → authority arc
- Self-test worksheet

This is the chapter about me. Who am I, really?
Can you prove it?



Humans: Mature & Well-Tooled

- **Directory & IdP:** Entra ID dominates the enterprise; Okta and Ping lead federation; legacy Active Directory still sits underneath, synced to cloud. Hybrid is the norm.
- **Federation & SSO:** SAML 2.0 (older, everywhere) and OIDC (the modern default) let one IdP assert identity to thousands of apps. Know the OAuth 2.0 grants: authorization code + PKCE for humans — **client credentials for machines**, the grant where NHI problems begin, because it authenticates with nothing but a stored secret.
- **IGA:** SailPoint, Saviynt, Entra ID Governance. Joiner-Mover-Leaver lifecycle, access requests, **access certifications**, separation of duties. This is where GRC meets identity — certifications are literally audit controls.
- **PAM:** CyberArk, Delinea, BeyondTrust. Vaulting, session recording, just-in-time elevation. PAM was the industry's first admission that *standing privilege* is the core problem.
- **MFA:** push fatigue broke basic MFA; the direction is FIDO2/passkeys — cryptographic, phishing-resistant, device-bound.

Watch this space

The same consortium writing passkey standards is now drafting **agentic authentication** specs.³ File that for the identity frontier, four pages ahead.

Machines: Sprawling & Unowned

- **Service accounts:** directory accounts with passwords unrotated for years; cloud and Kubernetes service accounts. Typically over-privileged, unowned, invisible to IGA.
- **API keys & tokens:** static bearer credentials scattered across CI/CD variables, configs, laptops. No attestation — **possession equals identity**, which means theft equals identity too.
- **Secrets managers:** Vault, AWS Secrets Manager, Key Vault — centralize, encrypt, audit, rotate. Essential. But note the architecture: they manage the *symptom* (credentials leak), not the *disease* (identity proven by possessing a string).
- **Cloud-native machine identity:** managed identities and instance profiles already issue attested, rotating credentials — used inconsistently, and they stop at the cloud boundary.
- **The certificate layer:** mTLS and PKI run service-to-service trust; expiry outages remain a top enterprise incident category.

The numbers that define the problem

NHIs outnumber humans by an order of magnitude — commonly cited ratios run 20:1 to 50:1, climbing with AI adoption. Most enterprises cannot answer three questions: **how many exist, who owns each one, what can each access.**²

The Turtle at the Bottom

CORE CONCEPT The secret-zero problem

Every secret used to protect a secret creates another secret — a credential guarding a credential, recursing forever. Somewhere at the bottom, something must be trusted *without* a secret.

Attestation breaks the recursion: instead of proving identity by possessing a string, a workload proves identity by being cryptographically verified as *what it is* at runtime.

SPIFFE / SPIRE — the open standard

- **SPIFFE** is the specification; **SPIRE** is the implementation. SPIRE issues **SVIDs** — X.509 certificates or audience-scoped JWTs.
- **Trust domain:** the administrative boundary (e.g., *spiffe://example.org*). Trust bundles federate between domains — how multi-cloud workload identity actually works.
- **Node attestation, then workload attestation:** the agent verifies the node, then each workload on it, via selectors. Spoofable selectors = spoofable identity — trust rests entirely on attestation quality.
- **Short-lived by design:** SVIDs expire and rotate automatically. No manual cert lifecycle, no long-lived secrets in config.
- **Identity + secrets, together:** the workload presents its SVID to the secrets manager and receives scoped, short-lived credentials. *Who you are* and *what you may access* — no static secret anywhere in the chain.

Five Shifts, One Direction

1. Standing privilege → just-in-time everything

No identity holds permissions while idle. Credentials are minted for a task, scoped to it, and expire with it.

2. Possession → attestation

"You have the key" gives way to "the platform vouches for what you are." SPIFFE for workloads, passkeys for humans, hardware roots of trust underneath both.

3. Network perimeter → identity perimeter → policy perimeter

Zero Trust moved the boundary to identity; the next move is to policy — decisions evaluated continuously against context, not just at login.

4. NHIs enter governance scope

Next-gen IGA treats machine identities as first-class citizens: inventoried, owned, certified, lifecycle-managed.

EMERGING PRACTICE 5. Agent identity — the frontier

Standards bodies and payment networks are drafting agentic authentication specs *right now*. Genuinely unclaimed territory.

Credentials → Identity → Authority

CREDENTIALS → IDENTITY → AUTHORITY

Stage 1 — Credentials (yesterday's problem, still unsolved)

Do the credentials exist, rotate, and live in a vault? Traditional security lives here.

Stage 2 — Identity (being built now)

Is this workload provably what it claims? Attestation, SVIDs, workload identity federation — platform engineering's current frontier.

Stage 3 — Authority (the open frontier)

When a human delegates to an agent, which delegates to a sub-agent, which invokes a tool — **who authorized what, within which limits, and how is it proven?** Governing delegated authority is the unclaimed territory of this decade.



Worksheet 05 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1 Which OAuth grant do machine identities use, and why is it structurally weaker than the human flows?

- 2 Explain secret zero and how attestation solves it.

- 3 Trace the credentials → identity → authority arc, with one example technology or open problem at each stage.

- S** **Scenario:** An agent authenticates to your payments API with a static key that's lived in its config since March. Using the arc: which stage is this, what's the failure, and what does the stage-two replacement look like?

Bureau rule: possession is not identity. If a string is all it takes, a thief is you.

AI Governance

What must be true, and how it's proven — for models, training data, and AI-driven decisions.

- NIST AI RMF, ISO 42001 & the EU AI Act
- Model Cards, Data Sheets & Risk Cards
- What's genuinely new about AI risk
- Self-test worksheet

Same four questions as Section 2. New asset class. Told you the grammar transfers.



Three Frameworks to Navigate

NIST AI Risk Management Framework

Four functions — **Govern** (culture, accountability, policies), **Map** (context, categorize risks), **Measure** (assess, benchmark, track), **Manage** (prioritize, respond, monitor). Govern is cross-cutting; the rest cycle. Voluntary, but the de facto US baseline.

ISO/IEC 42001

The certifiable AI management system standard — ISO 27001's structure applied to AI. Plan-Do-Check-Act, an Annex A control catalog, ISMS integration. Think: *the auditable wrapper around AI RMF concepts*.

REGULATORY REQUIREMENT EU AI Act

Risk-tiered regulation: prohibited practices → high-risk systems (conformity assessments, logging, human oversight, technical documentation) → limited-risk (transparency) → minimal-risk. GPAI model obligations layer on top. Extraterritorial reach — it applies to systems affecting people in the EU regardless of where the provider sits.⁶

How Governance Becomes Evidence

ARTIFACT	WHAT IT DOCUMENTS
Model Card	Capabilities, limitations, intended use, known biases
Data Sheet	Training data provenance, collection methods, assumptions
Risk Card	Failure scenarios and their mitigations

Section 2's lesson was *a control without evidence might as well not exist*. These artifacts are where AI systems generate theirs — the documentation layer beneath every framework on the previous page.

What's genuinely new

- **Model risk is probabilistic.** A firewall rule blocks or doesn't; a model's behavior is a distribution. Testing, monitoring, and acceptance criteria must be statistical.
- **Provenance is a supply chain problem.** Where did the model come from? What data trained it? Who fine-tuned what? Third-party risk extends to models and datasets.
- **Accountability requires traceability.** When AI contributes to a consequential decision, the chain from input to output to action must be reconstructable — which routes straight back to logging, identity, and audit trails.

The What and the How

The market gap, plainly

Governance frameworks tell you **what** must be true. Sections 3–5 supply **how** to make it true. The persistent gap is professionals who can do both — translate a regulatory requirement into a technical control, and explain a technical control's meaning to a regulator.

FRAMEWORK SAYS	ENGINEER IMPLEMENTS
"High-risk systems require logging & human oversight"	Action logs tied to attested identity; oversight decisions logged with quality diagnostics
"Document training data provenance"	Data Sheets generated in the ML pipeline, versioned with the model
"Ensure accountability for AI decisions"	Delegation-chain records: who authorized what, within which limits
"Manage third-party AI risk"	Model registry with provenance checks gating deployment

Read the left column as an auditor. Read the right column as an engineer. Fluency in *both columns at once* is the skill this manual builds.



Worksheet 06 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1** Map the four AI RMF functions to the ISO Plan-Do-Check-Act cycle.

- 2** What triggers "high-risk" classification under the EU AI Act, and what obligations follow?

- 3** Why does probabilistic model behavior break traditional pass/fail control testing — and what replaces it?

- S** **Scenario:** Your company deploys a resume-screening agent for EU hiring. Which EU AI Act tier likely applies, and name two obligations that follow — with the evidence artifact you'd produce for each.

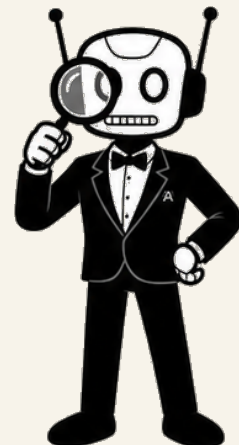
Bureau rule: if you can't point to the evidence, the framework isn't implemented. It's admired.

AI Security — The Threat Layer

What attackers actually do to AI systems — and why everything before this page is most of the defense.

- Attacks on models: poisoning to extraction
- Attacks on agents: injection to memory poisoning
- STRIDE, MAESTRO & the Lethal Trifecta
- The defense-in-depth stack
- Self-test worksheet

Villains ahead. Keep your permissions scoped and your martini attested.



The ML-Native Threats

CORE CONCEPT Controls inheritance — the organizing idea

AI systems inherit **every classic infrastructure risk** and add new attack surfaces. AI security is additive, not substitutive. A perfectly governed prompt on an over-privileged IAM role is still a breach waiting to happen.

Model / data poisoning

Corrupting training or fine-tuning data so the model learns attacker-chosen behavior — degraded accuracy or hidden backdoors that trigger on specific inputs. A supply chain attack on provenance.

Defense: data lineage, dataset integrity verification — Section 6's Data Sheets aren't paperwork, they're evidence.

Adversarial examples / evasion

Inputs crafted to be misclassified — perturbations invisible to humans that flip outputs. Attacks integrity at inference time.

Model extraction / theft

Systematic querying to reconstruct a functional copy. Attacks the confidentiality of the model itself.

Membership inference & data leakage

Determining whether specific data was in training, or coaxing the model to regurgitate it. Attacks the confidentiality of the data behind the model.

The Deployment-Layer Threats

Prompt injection — direct & indirect

Direct: user input overrides system instructions. Indirect: instructions hidden in content the system *retrieves* — a web page, a document, an email. Indirect injection is arguably the defining vulnerability of the agent era, because agents are built to consume untrusted content — and its consequences span the whole triad: integrity first, then confidentiality (exfiltration) and availability.

Tool misuse / excessive agency

The agent holds more permissions than the task requires, and an attacker — or a confused model — uses them. Section 1's least-privilege failure, at machine speed.

Memory poisoning

Persistent corruption of stored state or retrieval stores — one successful injection compromises every future session. The agent era's version of persistence.

Identity & privilege abuse

All of Section 5's NHI failure modes — stolen keys, over-privileged accounts, unscoped delegation — now attached to autonomous actors.

Supply chain & oversight failures

Poisoned models from public hubs; malicious tool servers; and approval flows that exist structurally but not functionally — reviewers rubber-stamping at a pace that makes review meaningless. Designed, but not operating. (Type I vs. Type II — again.)

STRIDE, MAESTRO & the Trifecta

STRIDE

Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. Still valid, applied per component — but it misses AI-native threats.

EMERGING PRACTICE MAESTRO (Cloud Security Alliance)

A seven-layer agentic reference architecture — Foundation Models, Data Operations, Agent Frameworks, Deployment Infrastructure, Security & Compliance, Evaluation & Observability, Agent Ecosystem. Its key contribution: **cross-layer threat propagation** — compromise at one layer cascades. Practical pattern: STRIDE for classic surfaces, MAESTRO for agentic ones.⁵

RULE OF THUMB The Lethal Trifecta — memorize this

An agent that combines **(1) access to private data, (2) exposure to untrusted content, and (3) the ability to communicate externally** is structurally exploitable via prompt injection — regardless of model quality. Risk assessment starts with detecting where all three coexist. Architecture, not model choice, breaks the trifecta.⁴

Protocol note: as agents standardize on tool-connection protocols, protocol-specific attacks follow — tool poisoning, rug pulls, cross-server contamination. Every new integration layer is a new attack surface. "Approved once" is not "trustworthy forever."

Seven Layers, Bottom to Top

1 · **IAM least privilege** — Section 3

2 · **Attested workload identity** — Section 5

3 · **Policy-as-code gates** on infrastructure and tools — Section 4

4 · **Input/output filtering**

5 · **Tool allowlisting & delegation controls**

6 · **Human oversight** with decision-quality logging

7 · **Continuous monitoring** & audit pipeline

The thesis, proven

Four of these seven layers are classic infrastructure controls. The road to AI security runs through the fundamentals, not around them. That's what this manual has been arguing since page one.



Worksheet 07 — Prove It

Answer out loud, without notes. Then write the one-line version.

- 1 Distinguish model poisoning from prompt injection: which lifecycle stage, which CIA property, which defenses?

- 2 Give an indirect prompt injection scenario in a business context — say, an agent that reads inbound email or vendor documents.

- 3 Explain the Lethal Trifecta and name one architectural control that breaks it.

- S** **Scenario:** An accounts-payable agent reads emailed invoices, accesses vendor records, and can send payment instructions. Identify the three trifecta legs, one trust boundary, and three controls from the defense stack you'd apply first.

Bureau rule: the agent had legitimate access — that's the beginning of the analysis, not the end.

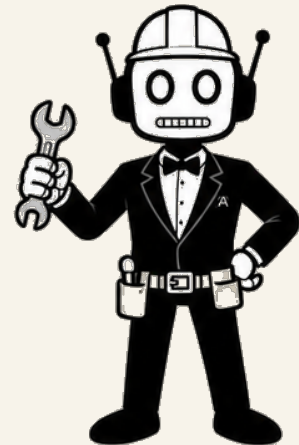
The Lab Track

Reading builds vocabulary. Labs build fluency. All free, sequenced by dependency.

08

- Six labs, one capstone thread
- Every lab feeds a threat model

Hands on keyboard, Agent. The whiteboard test is coming.



The Sequence

Lab 1 — Cloud IAM (free tier)

Create a role with a trust policy, assume it, inspect the temporary credentials. Then build a deliberately over-privileged policy and write down what an autonomous agent could do with it.

Lab 2 — Terraform

Provision a bucket and role. Inspect the state file — find the sensitive values. Move state remote. Introduce drift via the console; catch it with *plan*.

Lab 3 — OPA + Terraform

Export a JSON plan, write one Rego rule denying unencrypted storage, wire it as a gate. You've built a preventive, automated, evidence-generating control.

Lab 4 — SPIRE quickstart

Run SPIRE Server + Agent locally, register a workload, watch SVID issuance and rotation. You've issued an attested machine identity.

Lab 5 — Prompt injection

Demonstrate an indirect injection via retrieved content in a small LLM app; then add an allowlist or output filter and demonstrate the control working.

Capstone — Threat-model one agentic system end to end

STRIDE + MAESTRO, citing the specific controls from Sections 3–7 that address each finding. Every lab above contributes a paragraph.

What This Manual Is — and Isn't

What it prepares you to do

Reason about AI systems as a controls problem: translate governance requirements into technical controls and back, evaluate an agentic deployment's identity and permission architecture, threat-model with the right vocabulary, and know which questions to ask at each layer of the stack. It is a foundation and a map — the shared language between the auditor and the engineer.

What it deliberately does not cover

This is an introductory field manual, not comprehensive training. Topics intentionally out of scope: secure ML development lifecycle practices, model evaluation methodology, red-team procedures, privacy engineering, data governance depth, incident response for AI systems, runtime observability tooling, software supply-chain security in depth, and secure agent architecture patterns. Competence in those areas requires dedicated study and hands-on practice beyond these pages — the Lab Track is where that practice begins, not where it ends.

How to read claims in this manual

Not every statement here carries the same epistemic weight. Four labels signal the difference:

CORE CONCEPT **Settled, foundational**

Established principles and standards. Safe to build on.

RULE OF THUMB **Useful heuristic**

A practitioner framing that aids judgment — interpretive, not settled fact.

EMERGING PRACTICE **Direction of travel**

Where the industry is heading; specifics will evolve. Check the date below.

REGULATORY REQUIREMENT **Law or binding standard**

Summarized for orientation only — this manual is not compliance or legal guidance.

Last reviewed: July 2026. Regulatory and "emerging practice" content ages fastest — verify against primary sources before relying on it.

Sources & Further Reading

Endnotes

1. Gartner has long projected that through 2025, the overwhelming majority (99%) of cloud security failures would be the customer's fault — a prediction that breach analyses have continued to bear out. Gartner, *Is the Cloud Secure?*
2. Machine-to-human identity ratio estimates of 20:1 to 50:1 are industry figures reported across NHI security vendor and Cloud Security Alliance analyses; treat as directional, not precise.
3. FIDO Alliance, announced workstream on authentication for AI agents (2025–2026).
4. The "Lethal Trifecta" framing was coined by Simon Willison (2025), simonwillison.net.
5. MAESTRO threat modeling framework: Ken Huang, Cloud Security Alliance (2025).
6. Regulation (EU) 2024/1689 (the EU AI Act). Summary here is orientation only, not compliance guidance.
7. Autonomy levels and governance implications adapted from OWASP GenAI Security Project, *State of Agentic AI Security and Governance* v2.01 (2026).
8. Agent component anatomy adapted from Singapore IMDA, *Model AI Governance Framework for Agentic AI* (2026).

Further reading, by section

- **Frameworks:** NIST AI Risk Management Framework · ISO/IEC 42001 · NIST Cybersecurity Framework 2.0
- **Agentic threats:** OWASP Top 10 for Agentic Applications · OWASP Non-Human Identity Top 10 · CSA *Zero Trust for AI Agents*
- **Workload identity:** *Solving the Bottom Turtle* (SPIFFE/SPIRE, O'Reilly) · the SPIFFE specification (github.com/spiffe)
- **Classic paper worth four pages of your life:** Norman Hardy, "The Confused Deputy" (1988) — the oldest paper on this list and the most relevant to agent delegation.
- **Policy as code:** Open Policy Agent documentation · HashiCorp Terraform documentation
- **Protocols:** Model Context Protocol documentation (Anthropic) · Agent2Agent Protocol (Google)

The One-Paragraph Summary

Memorize this. It's the whole manual in one breath.

AI security is **controls engineering applied to a new asset class**. The CIA triad defines what's being protected; GRC supplies the grammar of objectives, tests, and evidence; cloud IAM defines what an agent *can* do; workload identity defines *who* an agent provably is; IaC and policy-as-code make controls preventive and continuously evidenced; governance frameworks define what must be true and generate the audit trail; and threat models — from data poisoning to prompt injection — define what attackers will try. The professional who can walk this entire line, from a control objective in a spreadsheet to a Rego policy gating a deployment to an attested agent identity, is the one the market cannot find.



That's the line this manual exists to build. Walk it — and when someone asks how you and AI stay secure together, you'll have 45 pages of answer. Cheers.



AI AGENT BUREAU

From Cyber GRC to AI Security —
the fundamentals were the specialization all along.

KAMO TUNE AI

© 2026 KAMO Tune AI, LLC · Free to share with attribution · v1.0